DESIGN OF DIGITAL SYSTEMS

ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

DEPARTMENT OF INFORMATION AND COMPUTER ENGINEERING

SEMESTER EXERCISE 2023

SIMPLE CIRCLE MIPS PROCESSOR

STUDENT DETAILS

NAME: ATHANASIOU VASILEIOS EVANGELOS
STUDENT ID: 19390005
STUDENT SEMESTER: 8th
STUDENT STATUS: UNDERGRADUATE
STUDY PROGRAM: UNIWA
THEORY DEPARTMENT: DEPARTMENT 2 KARKAZIS PANAGIOTIS
THEORY RESPONSIBLE: VOGIATZIS IOANNIS – KARKAZIS PANAGIOTIS
DELIVERY DATE: 15/9/2023

STUDENT PHOTO:

CONTENTS

1. Description of the circuit

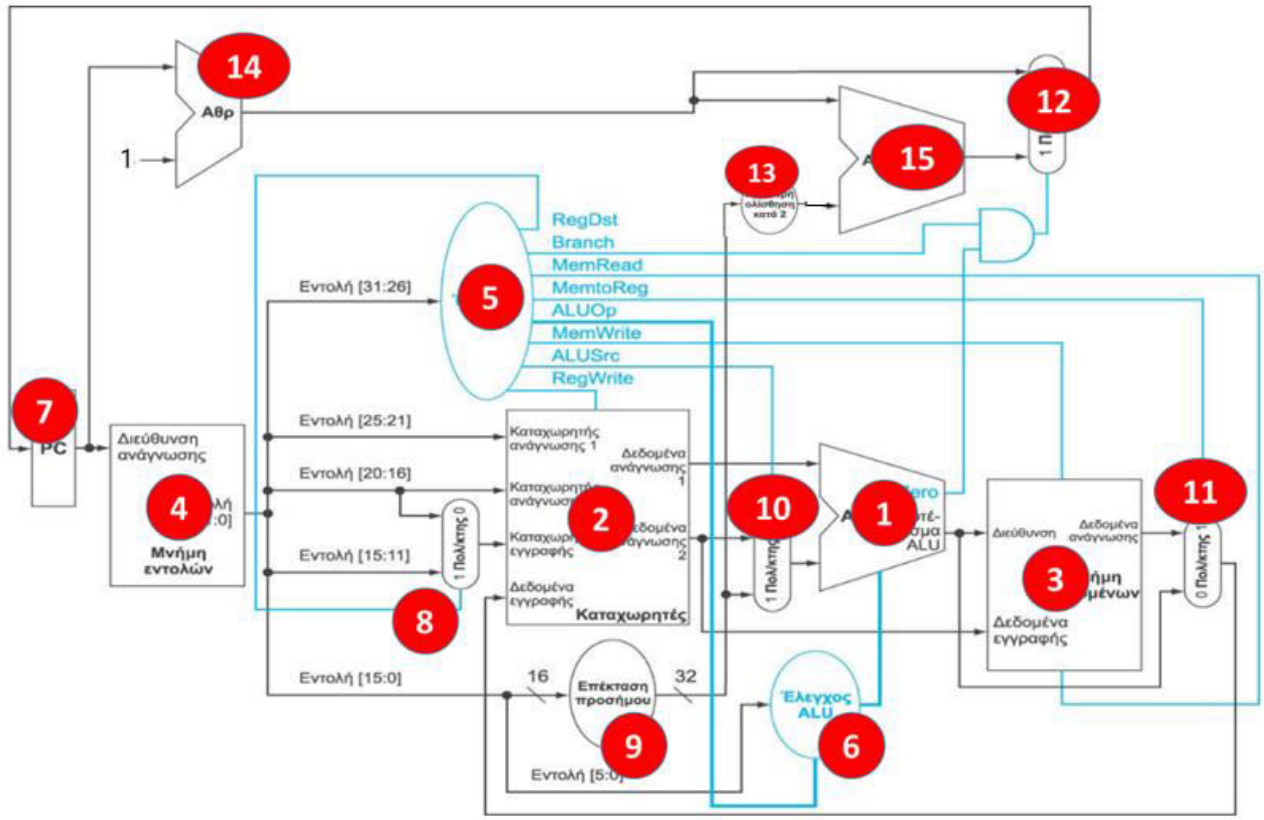


Figure 1. Single - cycle MIPS processor

1. ALU

The Arithmetic Logic Unit (ALU) accepts two inputs and performs arithmetic (addition, subtraction) and logic (AND , OR , XOR etc.) operations bit by bit . Which operation it will perform is determined by the output of the ALU Control Unit , which essentially sends a 2- bit control signal that signals the operation to be performed in the ALU .

If it is an arithmetic operation, then the output is “ ALU Result ”, otherwise, if it is a logical operation, the output is “ Zero ”.

2. Register File

The Register File contains the registers , where the data is stored and accessible for writing and reading. More specifically, it contains 2 read registers and one write register . It also contains 2 32- bit read data which is the output of the 2 read registers . Essentially the registers contain 5- bits that determine the number of the register , while the data contains the contents of the registers , hence the capacity of 32- bits of information.

Correspondingly, the same applies to recording, with the difference that recording only takes place when the " RegWrite " control signal of the Control Unit is 1.

3. Data Memory

The data memory contains a memory address and the data stored in a Register read register File . Also, there are 2 read and write control signals connected to the Control Unit, MemRead and MemWrite , respectively. The data memory output is used to write to the Register File , as long as the appropriate control signals have been activated by the Control Unit.

4. Memory of Commands

The instruction memory contains a memory address for input and the instruction corresponding to that address for output.

5. Control Unit

The control unit contains appropriate control signals to determine the corresponding functions that other processor components are attempting to perform.

ALU Control Unit

Every command that belongs to a certain category, e.g. type R (add , addi …) has a bit as well opcode , which is the main control signal for the ALU to know which instruction to execute.

7. PC

The program counter contains the address of the next instruction to be executed and is essentially a simple 32-bit register .

8. 5-fold 2-in-1 multiplexer

The multiplexer is a sequential circuit that, depending on the control signal, transmits the corresponding input to the output. Depending on the control signal RegDst of the Control Unit, the multiplexer transfers the corresponding 5- bit information from the command entered from the command memory, to the register of the Register File .

16-by-32 character extension unit

The sign extension unit converts the 16- bit number to 32- bit in order to add to the 32- bit adder .

10 . 32-way 2-in-1 multiplexer

This multiplexer transmits accordingly the " ALUSrc " control bit of the control unit, the corresponding information to the input of the ALU .

1 1. 32-way multiplexer 2-in-1

This multiplexer accordingly transmits the " MemtoReg " control bit of the control unit, the corresponding information in the data of the Register File .

12. 32-fold 2-in-1 multiplexer

This multiplexer forwards the Branch result accordingly AND Zero of the control unit and the ALU respectively, the corresponding information at the Program input Counter .

13. Shift Unit left by 2 (32- bits)

The shift unit shifts the information by 2- bits to the left, so that the overflow phenomenon does not occur in the addition.

14. Adder 32 bits

The adder adds the address contained in the program counter by 1 for the next command to be executed.

15. Adder 32 bits

The adder that adds the program address counter incremented by 1 with the information shifted 2 bits left.

2-input AND gate

AND gate which has as input the control signal Branch of the control unit and the output signal Zero of the ALU .

MIPS

The proper connection of the aforementioned components constitutes the single-cycle MIPS processor . Essentially, the operation of MIPS is described as follows:

1. Retrieve the address of the command from the program counter
2. Decoding the command from the command memory and transferring it to the registers of the Register File and at the entrance of the Control Unit
3. Carrying out the appropriate checks to transfer the information to the inputs of the ALU
4. Execution of the command
5. storage in memory
6. Calculation of the address of the next instruction to be executed.

2. Circuit code and testbench

1. ALU

alu.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all ;
USE ieee.std_logic_signed.all ;

ENTITY alu IS
PORT (
          operationSig : IN std_logic_vector (3 DOWNTO 0);
ALU_input1 : IN std_logic_vector (31 DOWNTO 0);
ALU_input2 : IN std_logic_vector (31 DOWNTO 0);
          ALU_output   : OUT std_logic_vector (31 DOWNTO 0);
          zeroSig      : OUT std_logic );
END alu ;

ARCHITECTURE behavioral OF alu IS
BEGIN
  PROCESS( operationSig )
VARIABLE tempInt : integer := 0;
BEGIN
CASE operationSig IS
WHEN "0001" => -- add
          zeroSig <= '0 ';
          tempInt : = to_integer (signed(ALU_input1) +
signed(ALU_input2));
          ALU_output <= std_logic_ vector ( to_signed ( tempInt , 32 ));
WHEN "0100" => -- add
          zeroSig <= '0 ';
          tempInt : = to_integer (signed(ALU_input1) +
signed(ALU_input2));
```

```
          ALU_output <= std_logic_ vector ( to_signed ( tempInt , 32 ));
WHEN "0101" => -- sub
          tempInt : = to_integer (signed(ALU_input1) -
signed(ALU_input2));
IF tempInt = 0 THEN
            zeroSig <= '1 ';
ELSE
            zeroSig <= '0 ';
END IF?
          ALU_output <= std_logic_ vector ( to_signed ( tempInt , 32 ));
WHEN "1000" => -- bne
          tempInt : = to_integer (signed(ALU_input1) -
signed(ALU_input2));
IF tempInt = 0 THEN
            zeroSig <= '1 ';
ELSE
            zeroSig <= '0 ';
END IF?
          ALU_output <= ALU_input1 XOR ALU_input2 ;
WHEN others =>
          zeroSig <= '0 ';
          ALU_output <= (others => '0' );
END CASE?
END PROCESS?
END behavioral?
```

alu_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY alu_tb IS
END alu_ tb ;

ARCHITECTURE testbench OF alu_tb IS
```

```
COMPONENT alu
PORT (
            operationSig : IN std_logic_vector (3 DOWNTO 0);
ALU_input1 : IN std_logic_vector (31 DOWNTO 0);
ALU_input2 : IN std_logic_vector (31 DOWNTO 0);
            ALU_output   : OUT std_logic_vector (31 DOWNTO 0);
            zeroSig      : OUT std_logic );
END COMPONENT;

SIGNAL operationSig_tb : std_logic_vector (3 DOWNTO 0);
, ALU_input2_tb, ALU_output_tb : std_logic_vector (31 DOWNTO 0);
SIGNAL zeroSig_ tb : std_logic ;

BEGIN
  Comp_Connection : alu PORT MAP ( operationSig => operationSig_tb,
ALU_input 1 = > ALU_input1_tb,
ALU_input 2 = > ALU_input2_tb,
                                  zeroSig => zeroSig_tb,
                                  ALU_output => ALU_output_tb );

PROCESS
BEGIN
-- add
    operationSig_tb <= "0001 ";
ALU_input1_tb <= "00000000000000000000000000001111 ";
ALU_input2_tb <= "00000000000000000000000000000001 ";
wait for 10 ns?

-- add
    operationSig_tb <= "0100 ";
ALU_input1_tb <= "00000000000000000000000000001111 ";
ALU_input2_tb <= "00000000000000000000000000000001 ";
```

```
wait for 10 ns?

-- sub (zero)
    operationSig_tb <= "0101 ";
ALU_input1_tb <= "00000000000000000000000000001111 ";
ALU_input2_tb <= "00000000000000000000000000001111 ";
wait for 10 ns?

-- sub ( non zero )
    operationSig_tb <= "0101 ";
ALU_input1_tb <= "00000000000000000000000000001111 ";
ALU_input2_tb <= "00000000000000000000000000000001 ";
wait for 10 ns?

-- bne (equal)
    operationSig_tb <= "1000 ";
ALU_input1_tb <= "00000000000000000000000000001111 ";
ALU_input2_tb <= "00000000000000000000000000001111 ";
wait for 10 ns?

-- bne (not equal)
    operationSig_tb <= "1000 ";
ALU_input1_tb <= "00000000000000000000000000001111 ";
ALU_input2_tb <= "00000000000000000000000000000001 ";
wait for 10 ns?

END PROCESS?
END testbench?
```

2. Register File

register_file.vhd

```
LIBRARY ieee ;
```

```
USE ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all ;
USE ieee.numeric_std.ALL ;

ENTITY register_file IS
PORT (
          RF_clock    : IN std_logic ;
          RF_reset    : IN std_logic ;
readReg1 : IN std_logic_vector (4 DOWNTO 0);
readReg2 : IN std_logic_vector (4 DOWNTO 0);
          writeRegIn  : IN std_logic_vector (4 DOWNTO 0);
          writeData   : IN std_logic_vector (31 DOWNTO 0);
          RegWrite    : IN std_logic ;
readData1 : OUT std_logic_vector (31 DOWNTO 0);
readData2 : OUT std_logic_vector (31 DOWNTO 0));
END register_file ;

ARCHITECTURE behavioral OF register_file IS
TYPE regArray IS ARRAY( 0 TO 15) OF std_logic_vector (31 DOWNTO 0);
SIGNAL register_file : regArray ;

BEGIN
  PROCESS( RF_clock , RegWrite , RF_reset )
BEGIN
IF rising_ edge ( RF_clock ) THEN
IF RegWrite = '1' THEN
        register_file ( to_integer (unsigned( writeRegIn ))) <= writeData
;
END IF?
END IF?
readData1 <= register_file ( to_integer (unsigned(readReg1)) );
readData2 <= register_file ( to_integer (unsigned(readReg2)) );
IF RF_reset = '1' THEN
readData1 <= (others => '0' );
```

```
readData2 <= (others => '0' );
END IF?
END PROCESS?
END behavioral?
```

register_file_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY register_file_tb IS
END register_file_tb ;

ARCHITECTURE testbench OF register_file_tb IS

COMPONENT register_file
PORT (
          RF_clock    : IN std_logic ;
          RF_reset    : IN std_logic ;
readReg1 : IN std_logic_vector (4 DOWNTO 0);
readReg2 : IN std_logic_vector (4 DOWNTO 0);
          writeRegIn  : IN std_logic_vector (4 DOWNTO 0);
          writeData   : IN std_logic_vector (31 DOWNTO 0);
          RegWrite    : IN std_logic ;
readData1 : OUT std_logic_vector (31 DOWNTO 0);
readData2 : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

SIGNAL readReg1_tb, readReg2_tb , writeRegIn_tb : std_logic_vector (4
DOWNTO 0);
SIGNAL writeData_tb , readData1_tb, readData2_ tb : std_logic_vector (31
DOWNTO 0);
SIGNAL RegWrite_tb , RF_clock_tb , RF_reset_tb : std_logic ;

BEGIN
```

```
  Comp_Connection : register_file PORT MAP ( RF_clock => RF_clock_tb,
                                             RF_reset => RF_reset_tb,
readReg1 => readReg1_tb,
readReg2 => readReg2_tb,
                                               writeRegIn =>
writeRegIn_tb,
                                               writeData => writeData_tb,
                                               RegWrite => RegWrite_tb,
readData1 => readData1_tb,
readData2 => readData2_tb );

clock_process : PROCESS
BEGIN
                    RF_clock_tb <= '0 ';
wait for 5 ns?
                    RF_clock_tb <= '1 ';
wait for 5 ns?
END PROCESS?

stim_ process : PROCESS
BEGIN
-- Perform write operation
                    RF_reset_tb <= '0 ';
                    writeRegIn_tb <= "00001 ";
                    writeData_tb <= "11110000111100001111000011110000 ";
                    RegWrite_tb <= '1 ';
WAIT FOR 10 ns?

-- Perform read operation
readReg1_tb <= "00001 ";
readReg2_tb <= "00010 ";
                  RegWrite_tb <= '0 ';
WAIT FOR 10 ns?
```

```
-- Reset the register file
                RF_reset_tb <= '1 ';
WAIT FOR 10 ns?
                RF_reset_tb <= '0 ';
                WAIT?

-- End the simulation
                WAIT?
END PROCESS?
END testbench?
```

3. Memory Data

data_memory.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all ;

ENTITY data_memory IS
PORT (
          writeData  : IN std_logic_vector (31 DOWNTO 0);
          addr       : IN std_logic_vector (31 DOWNTO 0);
          MemRead    : IN std_logic ;
          MemWrite   : IN std_logic ;
          readData   : OUT std_logic_vector (31 DOWNTO 0));
END data_ memory ;

ARCHITECTURE behavioral OF data_memory IS
TYPE regArray IS ARRAY( 0 TO 15) OF std_logic_vector (31 DOWNTO 0);
SIGNAL data_memory : regArray := (others => (others => '0'));
SIGNAL address : integer;

BEGIN
```

```
address <= to_integer ( unsigned( writeData )) WHEN
( to _integer (unsigned( writeData )) >= 0) ELSE 0;
    data_memory (address) <= writeData WHEN MemWrite = '1 ';
    readData <= data_memory (address) WHEN MemRead = '1 ';

END behavioral?
```

data_memory_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY data_memory_tb IS
END data_memory_ tb ;

ARCHITECTURE testbench OF data_memory_tb IS

COMPONENT data_memory
PORT (
          writeData  : IN std_logic_vector (31 DOWNTO 0);
          addr       : IN std_logic_vector (31 DOWNTO 0);
          MemRead    : IN std_logic ;
          MemWrite   : IN std_logic ;
          readData   : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

SIGNAL MemRead_tb , MemWrite_tb : std_logic ;
SIGNAL writeData_tb , addr_tb , readData_tb : std_logic_vector (31 DOWNTO
0);

BEGIN
  Comp_Connection : data_memory PORT MAP ( writeData => writeData_tb,
                                           addr => addr_tb,
                                           MemRead => MemRead_tb,
```

```
                                                MemWrite => MemWrite_tb,
                                                readData => readData_tb );
PROCESS
BEGIN

    writeData_tb <= (others => '0' );
    addr_tb <= (others => '0' );
    MemRead_tb <= '0 ';
    MemWrite_tb <= '0 ';
wait for 10 ns?

    writeData_tb <= "11110000111100001111000011110000 ";
    addr_tb <= "00000000000000000000000000000001 ";
    MemWrite_tb <= '1 ';
wait for 10 ns?

    addr_tb <= "00000000000000000000000000000001 ";
    MemRead_tb <= '1 ';
wait for 10 ns?

    writeData_tb <= "00001111000011110000111100001111 ";
    addr_tb <= "00000000000000000000000000000010 ";
    MemWrite_tb <= '1 ';
wait for 10 ns?

    addr_tb <= "00000000000000000000000000000010 ";
    MemRead_tb <= '1 ';
     wait for 10 ns?
END PROCESS?
END testbench?
```

4. Memory Commands

instructions_memory.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all ;

ENTITY instructions_memory IS
PORT (
          readAddr  : IN std_logic_vector (31 DOWNTO 0);
          instrOut   : OUT std_logic_vector (31 DOWNTO 0));
END instructions_ memory ;

ARCHITECTURE behavioral OF instructions_memory IS
TYPE regArray IS ARRAY( 0 TO 15) OF std_logic_vector (31 DOWNTO 0);
SIGNAL instructions_ memory : regArray := (
"00100000000000110000000000000001", -- addi $3, $0, 1
"00100000000001010000000000000011", -- addi $3, $0, 1
"00000000011000000011001000000000", -- L1: add $6, $3, $0
"10101100100001100000000000000000", -- sw $6, 0($4)
"00100000110001100000000000000001", -- addi $3, $3, 1
"00100001000010000000000000000001", -- addi $4, $4, 1
"00100000101001011111111111111111", -- addi $5, $5, -1
"00010100101000001111111111101010", -- bne $ 5.$ 0.L1
others => (others => '0')
);

SIGNAL instrAddr : integer;
SIGNAL instruction : std_logic_vector (31 DOWNTO 0);
BEGIN
  instrAddr <= to_integer (unsigned( readAddr )) WHEN ( to_integer
(unsigned( readAddr )) >= 0) ELSE 0;
instruction <= instructions_memory ( instrAddr ) WHEN ( instrAddr >= 0 )
ELSE std_logic_ vector ( to_signed (-1, 32));
  instrOut <= instruction;
END behavioral?
```

instructions_memory _tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY instructions_memory_tb IS
END instructions_memory_tb ;

ARCHITECTURE testbench OF instructions_memory_tb IS

COMPONENT instructions_memory
PORT (
          readAddr  : IN std_logic_vector (31 DOWNTO 0);
          instrOut  : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

SIGNAL readAddr_tb , instrOut_ tb : std_logic_vector (31 DOWNTO 0);

BEGIN
  Comp_Connection : instructions_memory PORT MAP ( readAddr =>
readAddr_tb,
                                                   instrOut => instrOut_tb );

PROCESS
BEGIN
    readAddr_tb <= "00000000000000000000000000000001"; -- Read address 1
wait for 10 ns?

    readAddr_tb <= "00000000000000000000000000000010"; -- Read address 2
wait for 10 ns?

    readAddr_tb <= "00000000000000000000000000000011"; -- Read address 3
wait for 10 ns?
END PROCESS?
```

```
END testbench?
```

5. Unit Control

control_unit.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY control_unit IS
PORT (
          CU_ clock : IN std_logic ;
          Instr   : IN std_logic_vector (5 DOWNTO 0);
          RegDst  : OUT std_logic ;
Branch : OUT std_logic ;
          MemRead : OUT std_logic ;
          MemtoReg : OUT std_logic ;
          ALUOp   : OUT std_logic_vector (1 DOWNTO 0);
          MemWrite : OUT std_logic ;
          ALUSrc  : OUT std_logic ;
          RegWrite : OUT std_logic );
END control_unit ;

ARCHITECTURE behavioral OF control_unit IS
BEGIN
PROCESS ( CU_clock , Instr )
BEGIN
IF rising_ edge ( CU_clock ) THEN
CASE Instr IS
-- add, sub
WHEN "000000" =>
        RegDst <= '1 ';
Branch <= '0 ';
        MemRead <= '0 ';
```

```
        MemtoReg <= '0 ';
        ALUOp <= "10 ";
        MemWrite <= '0 ';
        ALUSrc <= '0 ';
        RegWrite <= '1 ';
-- add
WHEN "001000" =>
        RegDst <= '0 ';
Branch <= '0 ';
        MemRead <= '0 ';
        MemtoReg <= '0 ';
        ALUOp <= "00 ";
        MemWrite <= '0 ';
        ALUSrc <= '1 ';
        RegWrite <= '1 ';
-- lw
WHEN "100011" =>
        RegDst <= '0 ';
Branch <= '0 ';
        MemRead <= '1 ';
        MemtoReg <= '1 ';
        ALUOp <= "00 ";
        MemWrite <= '0 ';
        ALUSrc <= '1 ';
        RegWrite <= '1 ';
      -- sw
      WHEN "101011" =>
        RegDst <= 'X ';
Branch <= '0 ';
        MemRead <= '0 ';
        MemtoReg <= 'X ';
        ALUOp <= "00 ";
        MemWrite <= '1 ';
```

```
        ALUSrc <= '1 ';
        RegWrite <= '0 ';
-- bne
WHEN "000101" =>
        RegDst <= 'X ';
Branch <= '1 ';
        MemRead <= '0 ';
        MemtoReg <= 'X ';
        ALUOp <= "01 ";
        MemWrite <= '0 ';
        ALUSrc <= '0 ';
        RegWrite <= '0 ';
WHEN others =>
        RegDst <= '0 ';
Branch <= '0 ';
        MemRead <= '0 ';
        MemtoReg <= '0 ';
        ALUOp <= "00 ";
        MemWrite <= '0 ';
        ALUSrc <= '0 ';
        RegWrite <= '0 ';
END CASE?
END IF?
END PROCESS?
END behavioral?
```

control_unit_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY control_unit_tb IS
END control_unit_tb ;
```

```
ARCHITECTURE testbench OF control_unit_tb IS


COMPONENT control_unit
PORT (
          CU_ clock : IN std_logic ;
          Instr   : IN std_logic_vector (5 DOWNTO 0);
          RegDst  : OUT std_logic ;
Branch : OUT std_logic ;
          MemRead : OUT std_logic ;
          MemtoReg : OUT std_logic ;
          ALUOp   : OUT std_logic_vector (1 DOWNTO 0);
          MemWrite : OUT std_logic ;
          ALUSrc  : OUT std_logic ;
          RegWrite : OUT std_logic );
END COMPONENT;

SIGNAL CU_clock_tb : std_logic ;
SIGNAL Instr_tb : std_logic_vector (5 DOWNTO 0);
SIGNAL ALUOp_ tb : std_logic_vector (1 DOWNTO 0);
SIGNAL RegDst_tb , Branch_tb , MemRead_tb , MemtoReg_tb ,
         MemWrite_tb , ALUSrc_tb , RegWrite_tb : std_logic ;

BEGIN
  Comp_Connection : control_unit PORT MAP ( CU_clock => CU_clock_tb,
                                            Instr => Instr_tb,
                                            RegDst => RegDst_tb,
Branch => Branch_tb ,
                                            MemRead => MemRead_tb,
                                            MemtoReg => MemtoReg_tb,
                                            ALUOp => ALUOp_tb,
                                            MemWrite => MemWrite_tb,
                                            ALUSrc => ALUSrc_tb,
```

```
                                                RegWrite => RegWrite_tb );

clock_process : PROCESS
BEGIN
  CU_clock_tb <= '0 ';
wait for 5 ns?
  CU_clock_tb <= '1 ';
wait for 5 ns?
END PROCESS?

stim_process : PROCESS
BEGIN
-- add, sub
    Instr_tb <= "000000 ";
wait for 10 ns?

-- add
    Instr_tb <= "001000 ";
wait for 10 ns?

-- lw
    Instr_tb <= "100011 ";
wait for 10 ns?

-- sw
    Instr_tb <= "101011 ";
wait for 10 ns?

-- bne
    Instr_tb <= "000101 ";
wait for 10 ns?

-- default
```

```
    Instr_tb <= "111111 ";
wait for 10 ns?
END PROCESS?
END testbench?
```

6. Unit ALU control

alu_control.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY alu_control IS
PORT (
          ALUOp   : IN std_logic_vector (1 DOWNTO 0);
          Instr   : IN std_logic_vector (5 DOWNTO 0);
          ALUCtrl : OUT std_logic_vector (3 DOWNTO 0));
END alu_control ;

ARCHITECTURE behavioral OF alu_control IS
BEGIN
PROCESS ( ALUOp , Instr )
BEGIN
IF ALUOp = "10" THEN -- addi
      ALUCtrl <= "0001 ";
ELSIF ALUOp = "01" THEN
      ALUCtrl <= "1000"; -- bne
ELSIF ALUOp = "00" THEN
CASE Instr IS
WHEN "100000" => -- add
          ALUCtrl <= "0100 ";
WHEN "100010" => -- sub
          ALUCtrl <= "0101 ";
WHEN others => -- default
```

```
          ALUCtrl <= "1110 ";
END CASE?
ELSE -- default
      ALUCtrl <= "1111 ";
END IF?
END PROCESS?
END behavioral?
```

alu _ control_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY alu_control_tb IS
END alu_control_tb ;

ARCHITECTURE testbench OF alu_control_tb IS

COMPONENT alu_control
PORT (
          ALUOp   : IN std_logic_vector (1 DOWNTO 0);
          Instr   : IN std_logic_vector (5 DOWNTO 0);
          ALUCtrl : OUT std_logic_vector (3 DOWNTO 0));
END COMPONENT;

SIGNAL ALUOp_ tb : std_logic_vector (1 DOWNTO 0);
SIGNAL Instr_tb : std_logic_vector (5 DOWNTO 0);
SIGNAL ALUCtrl_ tb : std_logic_vector (3 DOWNTO 0);

BEGIN
  Comp_Connection : alu_control PORT MAP ( ALUOp => ALUOp_tb,
                                           Instr => Instr_tb,
                                           ALUCtrl => ALUCtrl_tb );
```

```
PROCESS
BEGIN
      -- add
    ALUOp_tb <= "10 ";
    Instr_tb <= "001000 ";
wait for 10 ns?

-- bne
    ALUOp_tb <= "01 ";
    Instr_tb <= "000000 ";
wait for 10 ns?

-- add
    ALUOp_tb <= "00 ";
    Instr_tb <= "100000 ";
wait for 10 ns?

-- sub
    ALUOp_tb <= "00 ";
    Instr_tb <= "100010 ";
wait for 10 ns?

-- default
    ALUOp_tb <= "00 ";
    Instr_tb <= "111111 ";
wait for 10 ns?

-- default
    ALUOp_tb <= "11 ";
wait for 10 ns?
END PROCESS?
END testbench?
```

7 . PC

program_counter.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY program_counter IS
PORT (
          PC_ input : IN std_logic_vector (31 DOWNTO 0);
          PC_ clock : IN std_logic ;
          PC_ reset : IN std_logic ;
          PC_ output : OUT std_logic_vector (31 DOWNTO 0));
END program_ counter ;

ARCHITECTURE behavioral OF program_counter IS
BEGIN
  PROCESS( PC_reset , PC_clock )
BEGIN
IF PC_reset = '1' THEN
        PC_output <= "00000000000000000000000000000000 ";
END IF?
IF rising_ edge ( PC_clock ) THEN
        PC_output < = PC_input ;
END IF?
END PROCESS?
END behavioral?
```

program_counter_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY program_counter_tb IS
END program_counter_ tb ;
```

```
ARCHITECTURE testbench OF program_counter_tb IS

COMPONENT program_counter
PORT (
          PC_ input : IN std_logic_vector (31 DOWNTO 0);
          PC_ clock : IN std_logic ;
          PC_ reset : IN std_logic ;
          PC_ output : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

SIGNAL PC_clock_tb , PC_reset_tb : std_logic ;
SIGNAL PC_input_tb , PC_output_tb : std_logic_vector (31 DOWNTO 0);

BEGIN
Comp_Connection : program_counter PORT MAP ( PC_input => PC_input_tb,
                                             PC_clock => PC_clock_tb,
                                             PC_reset => PC_reset_tb,
                                             PC_output => PC_output_tb );
clock_process : PROCESS
BEGIN
                    PC_clock_tb <= '0 ';
wait for 5 ns?
                    PC_clock_tb <= '1 ';
wait for 5 ns?
END PROCESS?

stim_ process : PROCESS
     BEGIN
                    PC_reset_tb <= '1 ';
                    PC_input_tb <= "00000000000000000000000000000000 ";
wait for 10 ns?
```

```
                    PC_reset_tb <= '0 ';
wait for 10 ns?

                    PC_input_tb <= "00000000000000000000000000000001 ";
wait for 10 ns?

                    PC_input_tb <= "00000000000000000000000000000010 ";
wait for 10 ns?

                    PC_input_tb <= "00000000000000000000000000000011 ";
wait for 10 ns?
END PROCESS?
END testbench ;
```

8. 5-fold 2-in-1 multiplexer

mux_5x_2to1.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY mux_5x_2to1 IS
PORT (
Mux5_input1, Mux5_input 2 : IN std_logic_vector (4 DOWNTO 0);
Mux5_S : IN std_logic ;
Mux5_output : OUT std_logic_vector (4 DOWNTO 0));
END mux_5x_ 2to1;

ARCHITECTURE dataflow OF mux_5x_2to1 IS
BEGIN
```

```
Mux5_output <= Mux5_input1 WHEN Mux5_S = '1' ELSE
Mux5_input2 WHEN Mux5_S = '0 ';
END dataflow;
```

mux_5x_2to1_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY mux_5x_2to1_tb IS
END mux_5x_2to1_ tb;

ARCHITECTURE testbench OF mux_5x_2to1_tb IS

COMPONENT mux_5x_2to1
PORT (
Mux5_input1, Mux5_input 2 : IN std_logic_vector (4 DOWNTO 0);
Mux5_S : IN std_logic ;
Mux5_output : OUT std_logic_vector (4 DOWNTO 0));
END COMPONENT;

SIGNAL Mux5_input1_tb, Mux5_input2_tb, Mux5_output_tb : std_logic_vector
(4 DOWNTO 0);
SIGNAL Mux5_S_tb : std_logic ;

BEGIN
  Comp_ Connection : mux_5x_2to1 PORT MAP (Mux5_input1 => Mux5_input1_tb,
Mux5_input2 => Mux5_input2_tb,
Mux5_S => Mux5_S_tb,
Mux5_output => Mux5_output_tb );

PROCESS
BEGIN
Mux5_S_tb <= '0 ';
wait for 10 ns?
```

```
Mux5_S_tb <= '1 ';
wait for 10 ns?
END PROCESS?

PROCESS
BEGIN
Mux5_input1_tb <= "00000 ";
Mux5_input2_tb <= "11111 ";
wait for 10 ns?
END PROCESS?
END testbench ;
```

16-by-32 character extension unit

sign_extend_16to32.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY sign_extend_16to32 IS
PORT (
          SignExtend_ input : IN std_logic_vector (15 DOWNTO 0);
          SignExtend_ output : OUT std_logic_vector (31 DOWNTO 0));
END sign_extend_ 16to32;

ARCHITECTURE dataflow OF sign_extend_16to32 IS
SIGNAL ones : std_logic_vector (15 DOWNTO 0) := (OTHERS => '1');
SIGNAL zeros : std_logic_vector (15 DOWNTO 0) := (OTHERS => '0');
BEGIN
```

```
      SignExtend_output <= ones & SignExtend_input WHEN SignExtend_ input
( 15 ) = '1' ELSE
zeros & SignExtend_input WHEN SignExtend_input ( 15 ) = '0' ;
END dataflow;
```

sign _extend_16to32 _tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY sign_extend_16to32_tb IS
END sign_extend_16to32_ tb;

ARCHITECTURE testbench OF sign_extend_16to32_tb IS

COMPONENT sign_extend_ 16to32
PORT (
          SignExtend_ input : IN std_logic_vector (15 DOWNTO 0);
          SignExtend_ output : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

SIGNAL SignExtend_input_tb : std_logic_vector (15 DOWNTO 0);
SIGNAL SignExtend_output_tb : std_logic_vector (31 DOWNTO 0);

BEGIN
  Comp_ Connection : sign_extend_16to32 PORT MAP ( SignExtend_input =>
SignExtend_input_tb,
                                                   SignExtend_output =>
SignExtend_output_tb );

PROCESS
BEGIN
    SignExtend_input_tb <= "0101010101010101 ";
wait for 10 ns?
```

```
    SignExtend_input_tb <= "1010101010101010 ";
wait for 10 ns?
END PROCESS?
END testbench?
```

10, 11, 12. 32- pl 2- in -1 multiplexer

mux_32x_2to1.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY mux_32x_2to1 IS
PORT (
Mux32_input1, Mux32_input 2 : IN std_logic_vector (31 DOWNTO 0);
Mux32_S : IN std_logic ;
Mux32_output : OUT std_logic_vector (31 DOWNTO 0));
END mux_32x_ 2to1;

ARCHITECTURE dataflow OF mux_32x_2to1 IS
BEGIN
Mux32_output <= Mux32_input1 WHEN Mux32_S = '1' ELSE
Mux32_input2 WHEN Mux32_S = '0 ';
END dataflow;
```

mux_32x_2to1_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY mux_32x_2to1_tb IS
END mux_32x_2to1_ tb;

ARCHITECTURE testbench OF mux_32x_2to1_tb IS
```

```
COMPONENT mux_32x_2to1

PORT (

Mux32_input1, Mux32_input 2 : IN std_logic_vector (31 DOWNTO 0);

Mux32_S : IN std_logic ;

Mux32_output : OUT std_logic_vector (31 DOWNTO 0));

END COMPONENT;


SIGNAL Mux32_input1_tb, Mux32_input2_tb, Mux32_output_tb :
std_logic_vector (31 DOWNTO 0);

SIGNAL Mux32_S_ tb : std_logic ;


BEGIN

  Comp_ Connection : mux_32x_2to1 PORT MAP (Mux32_input1 =>
Mux32_input1_tb,

Mux32_input2 => Mux32_input2_tb,

Mux32_S => Mux32_S_tb,

Mux32_output => Mux32_output_tb );


PROCESS

BEGIN

Mux32_S_tb <= '0 ';

wait for 10 ns?

Mux32_S_tb <= '1 ';

wait for 10 ns?

END PROCESS?


PROCESS

BEGIN

Mux32_input1_tb <= "00000000000000000000000000000000 ";

Mux32_input2_tb <= "11111111111111111111111111111111 ";

wait for 10 ns?

END PROCESS?

END testbench ;
```

13. Shift Unit left by 2 (32- bits)

shifter_2left.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all ;

ENTITY shifter_2left IS
PORT (
          shifter_input  : IN std_logic_vector (31 DOWNTO 0);
          shifter_output : OUT std_logic_vector (31 DOWNTO 0));
END shifter_ 2left;

ARCHITECTURE behavioral OF shifter_2left IS
SIGNAL tmp : unsigned(31 DOWNTO 0);
BEGIN
  tmp <= to_unsigned ( to_integer ( signed( Shifter_input )), tmp'length
) SLL 2;
  Shifter_output <= std_logic_vector ( to_signed ( to_integer ( tmp ),
Shifter_output'length ) );
END behavioral?
```

shifter_2left_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY shifter_2left_tb IS
END shifter_2left_ tb;

ARCHITECTURE testbench OF shifter_2left_tb IS

COMPONENT shifter_2left
PORT (
          shifter_input  : IN std_logic_vector (31 DOWNTO 0);
```

```
          shifter_output : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;


SIGNAL Shifter_input_tb , Shifter_output_tb : std_logic_vector (31 DOWNTO
0);


BEGIN
  Comp_ Connection : shifter_2left PORT MAP ( Shifter_input =>
Shifter_input_tb,
                                              Shifter_output =>
Shifter_output_tb );
PROCESS
BEGIN
    Shifter_input_tb <= (others => '0' );
wait for 10 ns?


    Shifter_input_tb <= (others => '1' );
wait for 10 ns?


    Shifter_input_tb <= "11001100110011001100110011001100 ";
wait for 10 ns?
END PROCESS?
END testbench?
```

14. Adder 32 bits

pc_adder.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all ;


ENTITY pc_adder IS
PORT (
          previousCmdAddr : IN std_logic_vector (31 DOWNTO 0);
```

```
          nextCmdAddr    : OUT std_logic_vector (31 DOWNTO 0));
END pc_adder ;

ARCHITECTURE dataflow OF pc_adder IS
SIGNAL sum : unsigned(31 DOWNTO 0);
BEGIN
sum <= unsigned( previousCmdAddr ) + 1;
    nextCmdAddr <= std_logic_vector (sum );
END dataflow;
```

pc_adder_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY pc_adder_tb IS
END pc_adder_ tb ;

ARCHITECTURE testbench OF pc_adder_tb IS

COMPONENT pc_adder
PORT (
          previousCmdAddr : IN std_logic_vector (31 DOWNTO 0);
          nextCmdAddr    : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

SIGNAL previousCmdAddr_tb , nextCmdAddr_tb : std_logic_vector (31 DOWNTO
0);

BEGIN
  Comp_Connection : pc_adder PORT MAP ( previousCmdAddr =>
previousCmdAddr_tb,
                                        nextCmdAddr => nextCmdAddr_tb
);
```

```
PROCESS
BEGIN
    previousCmdAddr_tb <= "00000000000000000000000000000000 ";
wait for 10 ns?

    previousCmdAddr_tb <= "00000000000000000000000000000001 ";
wait for 10 ns?

    previousCmdAddr_tb <= "00000000000000000000000000000010 ";
wait for 10 ns?

    previousCmdAddr_tb <= "00000000000000000000000000000011 ";
wait for 10 ns?

    previousCmdAddr_tb <= "11111111111111111111111111111111 ";
wait for 10 ns?

    previousCmdAddr_tb <= "11111111111111111111111111111111 ";
wait for 10 ns?
END PROCESS?
END testbench?
```

15. Adder 32 bits

fullAdder_32bit.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all ;
USE ieee.numeric_std.all ;

ENTITY fullAdder_32bit IS
PORT (
Adder_input1, Adder_input 2 : IN std_logic_vector (31 DOWNTO 0);
```

```
          Adder_output              : OUT std_logic_vector (31 DOWNTO 0)
);
END fullAdder_ 32bit;

ARCHITECTURE dataflow OF fullAdder_32bit IS
BEGIN
    Adder_output <= Adder_input1 + Adder_input2 ;
END dataflow;
```

fullAdder_32bit_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY fullAdder_32bit_tb IS
END fullAdder_32bit_ tb;

ARCHITECTURE testbench OF fullAdder_32bit_tb IS

COMPONENT fullAdder_32bit
PORT (
Adder_input1, Adder_input 2 : IN std_logic_vector (31 DOWNTO 0);
          Adder_Cin                 : IN std_logic_vector (0 DOWNTO 0);
          Adder_output              : OUT std_logic_vector (31 DOWNTO 0);
          Adder_Cout                : OUT std_logic );
END COMPONENT;

, Adder_input2_tb, Adder_output_tb : std_logic_vector (31 DOWNTO 0);
SIGNAL Adder_Cin_tb : std_logic_vector (0 DOWNTO 0);
SIGNAL Adder_Cout_ tb : std_logic ;

BEGIN
  Comp_ Connection : fullAdder_32bit PORT MAP (Adder_input1 =>
Adder_input1_tb,
Adder_input2 => Adder_input2_tb,
```

```
                                                    Adder_Cin => Adder_Cin_tb,
                                                    Adder_output =>
Adder_output_tb,
                                                    Adder_Cout => Adder_Cout_tb
);

PROCESS
BEGIN
Adder_input1_tb <= "01010101010101010101010101010101 ";
Adder_input2_tb <= "00110011001100110011001100110011 ";
    Adder_Cin_tb <= "1 ";
wait for 10 ns?

Adder_input1_tb <= "11000011001100110011001100110011 ";
Adder_input2_tb <= "01111111111111111111111111111111 ";
    Adder_Cin_tb <= "0 ";
wait for 10 ns?
END PROCESS?
END testbench?
```

input AND gate

and2_gate.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY and_2gate IS
PORT (
Branch, Zero : IN std_logic ;
          AND_output  : OUT std_logic );
END and_ 2gate;

ARCHITECTURE dataflow OF and_2gate IS
BEGIN
```

```
      AND_output <= Branch AND Zero;
END dataflow;
```

and2_gate_tb.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY and_2gate_tb IS
END and_2gate_ tb;

ARCHITECTURE testbench OF and_2gate_tb IS

COMPONENT and_2gate
PORT (
Branch, Zero : IN std_logic ;
      AND_output            : OUT std_logic );
END COMPONENT;

SIGNAL Branch_tb , Zero_tb , AND_output_tb : std_logic ;

BEGIN
  Comp_ Connection : and_2gate PORT MAP (Branch => Branch_tb ,
Zero => Zero_tb ,
                                        AND_output => AND_output_tb );

PROCESS
BEGIN
    Branch_tb <= '0'; Zero_tb <= '0 ';
wait for 10 ns?
    Branch_tb <= '1'; Zero_tb <= '0 ';
```

```
wait for 10 ns?
    Branch_tb <= '0'; Zero_tb <= '1 ';
wait for 10 ns?
    Branch_tb <= '1'; Zero_tb <= '1 ';
wait for 10 ns?
END PROCESS?
END testbench?
```

MIPS

19390005_ATHANASIOU_02_MIPS.vhd

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_signed.all ;

ENTITY MIPS IS
PORT (
          MIPS_ clock : IN std_logic ;
          MIPS_ reset : IN std_logic ;
          PC_output_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
          ALU_output_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
          zeroSig_MIPS  : OUT std_logic ;
readData1_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
readData2_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
          Branch_MIPS  : OUT std_logic ;
          instrOut_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
          writeData_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
          MemWrite_ MIPS : OUT std_logic ;
          RegWrite_ MIPS : OUT std_logic
);
END MIPS?

ARCHITECTURE structural OF MIPS IS
```

```
COMPONENT alu
PORT (
  operationSig : IN std_logic_vector (3 DOWNTO 0);
ALU_input1 : IN std_logic_vector (31 DOWNTO 0);
ALU_input2 : IN std_logic_vector (31 DOWNTO 0);
  ALU_output   : OUT std_logic_vector (31 DOWNTO 0);
  zeroSig      : OUT std_logic );
END COMPONENT;

COMPONENT register_file
PORT (
  RF_clock    : IN std_logic ;
  RF_reset    : IN std_logic ;
readReg1 : IN std_logic_vector (4 DOWNTO 0);
readReg2 : IN std_logic_vector (4 DOWNTO 0);
  writeRegIn  : IN std_logic_vector (4 DOWNTO 0);
  writeData   : IN std_logic_vector (31 DOWNTO 0);
  RegWrite    : IN std_logic ;
readData1 : OUT std_logic_vector (31 DOWNTO 0);
readData2 : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

COMPONENT data_memory
PORT (
  writeData  : IN std_logic_vector (31 DOWNTO 0);
  addr       : IN std_logic_vector (31 DOWNTO 0);
  MemRead    : IN std_logic ;
  MemWrite   : IN std_logic ;
  readData   : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

COMPONENT instructions_memory
```

```
PORT (
  readAddr  : IN std_logic_vector (31 DOWNTO 0);
  instrOut   : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

COMPONENT control_unit
PORT (
  CU_ clock : IN std_logic ;
  Instr   : IN std_logic_vector (5 DOWNTO 0);
  RegDst  : OUT std_logic ;
Branch : OUT std_logic ;
  MemRead : OUT std_logic ;
  MemtoReg : OUT std_logic ;
  ALUOp   : OUT std_logic_vector (1 DOWNTO 0);
  MemWrite : OUT std_logic ;
  ALUSrc  : OUT std_logic ;
  RegWrite : OUT std_logic );
END COMPONENT;

COMPONENT alu_control
PORT (
  ALUOp   : IN std_logic_vector (1 DOWNTO 0);
  Instr   : IN std_logic_vector (5 DOWNTO 0);
  ALUCtrl : OUT std_logic_vector (3 DOWNTO 0));
END COMPONENT;

COMPONENT program_counter
PORT (
  PC_ input : IN std_logic_vector (31 DOWNTO 0);
  PC_ clock : IN std_logic ;
  PC_ reset : IN std_logic ;
  PC_ output : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;
```

```
COMPONENT pc_adder
PORT (
  previousCmdAddr : IN std_logic_vector (31 DOWNTO 0);
  nextCmdAddr   : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;
COMPONENT mux_5x_2to1
PORT (
Mux5_input1, Mux5_input 2 : IN std_logic_vector (4 DOWNTO 0);
Mux5_S : IN std_logic ;
Mux5_output : OUT std_logic_vector (4 DOWNTO 0));
END COMPONENT;

COMPONENT sign_extend_ 16to32
PORT (
  SignExtend_input : IN std_logic_vector (15 DOWNTO 0);
  SignExtend_ output : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

COMPONENT mux_32x_2to1
PORT (
Mux32_input1, Mux32_input 2 : IN std_logic_vector (31 DOWNTO 0);
Mux32_S : IN std_logic ;
Mux32_output : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

COMPONENT shifter_2left
PORT (
  shifter_input  : IN std_logic_vector (31 DOWNTO 0);
  shifter_output : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;

COMPONENT fullAdder_32bit
```

```
PORT (
Adder_input1, Adder_input 2 : IN std_logic_vector (31 DOWNTO 0);
  Adder_output              : OUT std_logic_vector (31 DOWNTO 0));
END COMPONENT;
COMPONENT and_2gate
PORT (
Branch, Zero : IN std_logic ;
  AND_output    : OUT std_logic );
END COMPONENT;

-- ALU out signals
SIGNAL ALU_output_ Sig : std_logic_vector (31 DOWNTO 0);
SIGNAL zeroSig_Sig  : std_logic ;

-- Register File out signals
SIGNAL readData1_ Sig : std_logic_vector (31 DOWNTO 0);
SIGNAL readData2_ Sig : std_logic_vector (31 DOWNTO 0);
SIGNAL writeData_ Sig : std_logic_vector (31 DOWNTO 0);

-- Data Memory out signals
SIGNAL readDataMemory_ Sig : std_logic_vector (31 DOWNTO 0);

-- Instructions Memory out signals
SIGNAL instrOut_ Sig : std_logic_vector (31 DOWNTO 0);

-- Control Unit
SIGNAL Instr_Sig   : std_logic_vector (5 DOWNTO 0);
SIGNAL RegDst_Sig  : std_logic ;
SIGNAL Branch_Sig  : std_logic ;
SIGNAL MemRead_Sig : std_logic ;
SIGNAL MemtoReg_ Sig : std_logic ;
SIGNAL ALUOp_Sig   : std_logic_vector (1 DOWNTO 0);
SIGNAL MemWrite_ Sig : std_logic ;
```

```
SIGNAL ALUSrc_Sig  : std_logic ;
SIGNAL RegWrite_ Sig : std_logic ;

-- ALU Control out signals
SIGNAL ALUCtrl_ Sig : std_logic_vector (3 DOWNTO 0);

-- Program Counter out signals
SIGNAL PC_output_ Sig : std_logic_vector (31 DOWNTO 0);

-- PC Adder out signals
SIGNAL nextCmdAddr_ Sig : std_logic_vector (31 DOWNTO 0);

-- Mux 5x 2-to-1 out signals
SIGNAL Mux5_output_ Register : std_logic_vector (4 DOWNTO 0);

-- Sign Extend 16-to-32 out signals
SIGNAL SignExtend_output_ Sig : std_logic_vector (31 DOWNTO 0);

-- Mux 32x 2-to-1 out signals
SIGNAL Mux32_output_PC : std_logic_vector (31 DOWNTO 0);
SIGNAL Mux32_output_ALU : std_logic_vector (31 DOWNTO 0);
SIGNAL Mux32_output_ Register : std_logic_vector (31 DOWNTO 0);

-- Shifter 2 Left out signals
SIGNAL Shifter_output_ Sig : std_logic_vector (31 DOWNTO 0);

-- Full Adder 32- bit out signals
SIGNAL Adder_output_ Sig : std_logic_vector (31 DOWNTO 0);
SIGNAL Adder_Cout_Sig : std_logic ;

-- AND 2-gate out signals
SIGNAL AND_output_ Sig : std_logic ;
```

```
BEGIN


PC_output_MIPS   <= PC_output_ Sig ;
instrOut_MIPS    <= instrOut_ Sig ;
ALU_output_MIPS  <= ALU_output_ Sig ;
readData1_MIPS   <= readData1_ Sig;
readData2_MIPS   <= readData2_ Sig;
writeData_ MIPS       <= readDataMemory_Sig WHEN MemtoReg_Sig = '1' ELSE
ALU_output_Sig ;
Branch_MIPS           < = Branch_Sig ;
zeroSig_MIPS          <= zeroSig_ Sig ;
RegWrite_MIPS    <= RegWrite_ Sig ;
MemWrite_MIPS    <= MemWrite_ Sig ;


PCAdder : pc_adder PORT MAP ( previousCmdAddr => PC_output_Sig ,
                              nextCmdAddr => nextCmdAddr_Sig );


BranchAdder : fullAdder_32bit PORT MAP (Adder_input1 => nextCmdAddr_Sig ,
Adder_input2 => Shifter_output_Sig ,
                                       Adder_output => Adder_output_Sig
);


ProgramCounter : program_counter PORT MAP ( PC_input => Mux32_output_PC,
                                            PC_clock => MIPS_clock,
                                            PC_reset => MIPS_reset,
                                            PC_output => PC_output_Sig );


InstructionsMemory : instructions_memory PORT MAP ( readAddr =>
PC_output_Sig,
                                                    instrOut =>
instrOut_Sig );


RegisterFile : register_file PORT MAP ( RF_clock => MIPS_clock,
                                        RF_reset => MIPS_reset,
readReg1 => instrOut_ Sig ( 25 DOWNTO 21),
```

```
readReg2 => instrOut_ Sig ( 20 DOWNTO 16),
                                        writeRegIn =>
Mux5_output_Register,
                                        writeData =>
Mux32_output_Register,
                                        RegWrite => RegWrite_Sig,
readData1 => readData1_Sig,
readData2 => readData2_Sig );

Control Unit : control_unit PORT MAP ( CU_clock => MIPS_clock,
                                       Instr => instrOut_ Sig ( 31 DOWNTO
26),
                                       RegDst => RegDst_Sig,
Branch => Branch_Sig ,
                                       MemRead => MemRead_Sig,
                                       MemtoReg => MemtoReg_Sig,
                                       ALUOp => ALUOp_Sig,
                                       MemWrite => MemWrite_Sig,
                                       ALUSrc => ALUSrc_Sig,
                                       RegWrite => RegWrite_Sig );

ALUControl : alu_control PORT MAP ( ALUOp => ALUOp_Sig,
                                    Instr => instrOut_ Sig ( 5 DOWNTO 0 ),
                                    ALUCtrl => ALUCtrl_Sig );

ArithmeticLogicUnit : alu PORT MAP ( operationSig => ALUCtrl_Sig,
ALU_input1 => readData1_Sig,
ALU_input2 => Mux32_output_ALU,
                                     ALU_output => ALU_output_Sig,
                                     zeroSig => zeroSig_Sig );

DataMemory : data_memory PORT MAP ( writeData => readData2_Sig,
                                    addr => ALU_output_Sig,
                                    MemRead => MemRead_Sig,
```

```
                                        MemWrite => MemWrite_Sig,
                                        readData => readDataMemory_Sig );

Mux5x2to1_ Register : mux_5x_2to1 PORT MAP (Mux5_input1 => instrOut_Sig
(20 DOWNTO 16),
Mux5_input2 => instrOut_ Sig ( 15 DOWNTO 11),
Mux5_S => RegDst_Sig ,
Mux5_output => Mux5_output_Register );

Mux32x2to1_ ALU : mux_32x_2to1 PORT MAP (Mux32_input1 => readData2_Sig,
Mux32_input2 => SignExtend_output_Sig ,
Mux32_S => ALUSrc_Sig ,
Mux32_output => Mux32_output_ALU );

Mux32x2to1_ Register : mux_32x_2to1 PORT MAP (Mux32_input1 =>
readDataMemory_Sig ,
Mux32_input2 => ALU_output_Sig ,
Mux32_S => MemtoReg_Sig ,
Mux32_output => Mux32_output_Register );

Mux32x2to1_ PC : mux_32x_2to1 PORT MAP (Mux32_input1 => nextCmdAddr_Sig ,
Mux32_input2 => Adder_output_Sig ,
Mux32_S => AND_output_Sig ,
Mux32_output => Mux32_output_PC );

SignExtend16to 32 : sign_extend_16to32 PORT MAP ( SignExtend_input =>
instrOut_Sig (15 DOWNTO 0),
                                                SignExtend_output =>
SignExtend_output_Sig );

AND2 gate : and_2gate PORT MAP (Branch => Branch_Sig ,
Zero => zeroSig_Sig ,
                                AND_output => AND_output_Sig );
```

```
Shifter2 Left : shifter_2left PORT MAP ( Shifter_input =>
SignExtend_output_Sig,
                                        Shifter_output =>
Shifter_output_Sig );


END structural;
```

19390005_ATHANASIOU_03_testbench.vhd

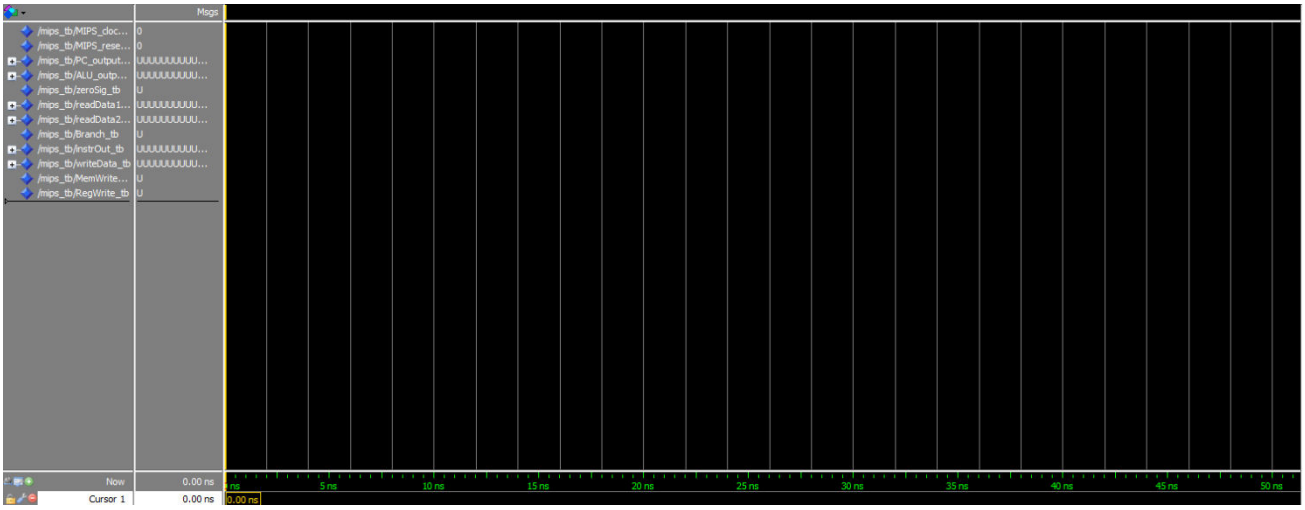
```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;


ENTITY MIPS_tb IS
END MIPS_ tb ;


ARCHITECTURE testbench OF MIPS_tb IS


MIPS COMPONENTS
PORT (
          MIPS_ clock : IN std_logic ;
          MIPS_ reset : IN std_logic ;
          PC_output_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
          ALU_output_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
          zeroSig_MIPS  : OUT std_logic ;
readData1_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
readData2_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
          Branch_MIPS  : OUT std_logic ;
          instrOut_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
          writeData_ MIPS : OUT std_logic_vector (31 DOWNTO 0);
          MemWrite_ MIPS : OUT std_logic ;
          RegWrite_ MIPS : OUT std_logic
);
END COMPONENT;


SIGNAL MIPS_clock_tb       : std_logic := '0';
SIGNAL MIPS_reset_tb       : std_logic := '0';
```

```
SIGNAL PC_output_tb        : std_logic_vector (31 DOWNTO 0) := (others =>
'0');
SIGNAL ALU_output_tb       : std_logic_vector (31 DOWNTO 0) := (others =>
'0');
SIGNAL zeroSig_tb          : std_logic := '0';
SIGNAL readData1_tb : std_logic_vector (31 DOWNTO 0) := (others => '0');
SIGNAL readData2_tb : std_logic_vector (31 DOWNTO 0) := (others => '0');
SIGNAL Branch_tb           : std_logic := '0';
SIGNAL instrOut_ tb : std_logic_vector (31 DOWNTO 0) := (others => '0');
SIGNAL writeData_tb   : std_logic_vector (31 DOWNTO 0) := (others =>
'0');
SIGNAL MemWrite_tb    : std_logic := '0';
SIGNAL RegWrite_tb        : std_logic := '0';


BEGIN
MIPS_testbench : MIPS PORT MAP (
  MIPS_clock => MIPS_clock_tb ,
  MIPS_reset => MIPS_reset_tb ,
  PC_output_MIPS => PC_output_tb ,
  ALU_output_MIPS => ALU_output_tb ,
  zeroSig_MIPS => zeroSig_tb ,
readData1_MIPS => readData1_tb,
readData2_MIPS => readData2_tb,
  Branch_MIPS => Branch_tb ,
  instrOut_MIPS => instrOut_tb ,
  writeData_MIPS => writeData_tb ,
  MemWrite_MIPS => MemWrite_tb ,
  RegWrite_MIPS => RegWrite_tb
);


clock_process : PROCESS
BEGIN
      MIPS_clock_tb <= '0 ';
wait for 5 ns?
```

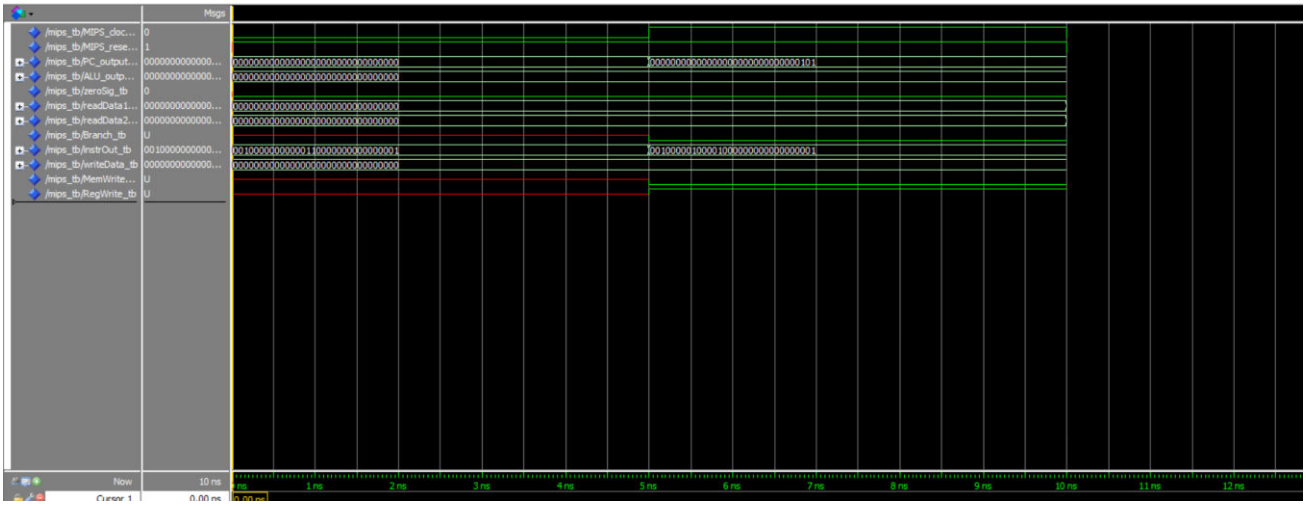
```
      MIPS_clock_tb <= '1 ';
wait for 5 ns?
END PROCESS?

reset_process : PROCESS
BEGIN
        MIPS_reset_tb <= '1 ';
wait for 10 ns?
        MIPS_reset_tb <= '0 ';
        wait?
END PROCESS?
END testbench?
```

3. Screenshot from her implementation

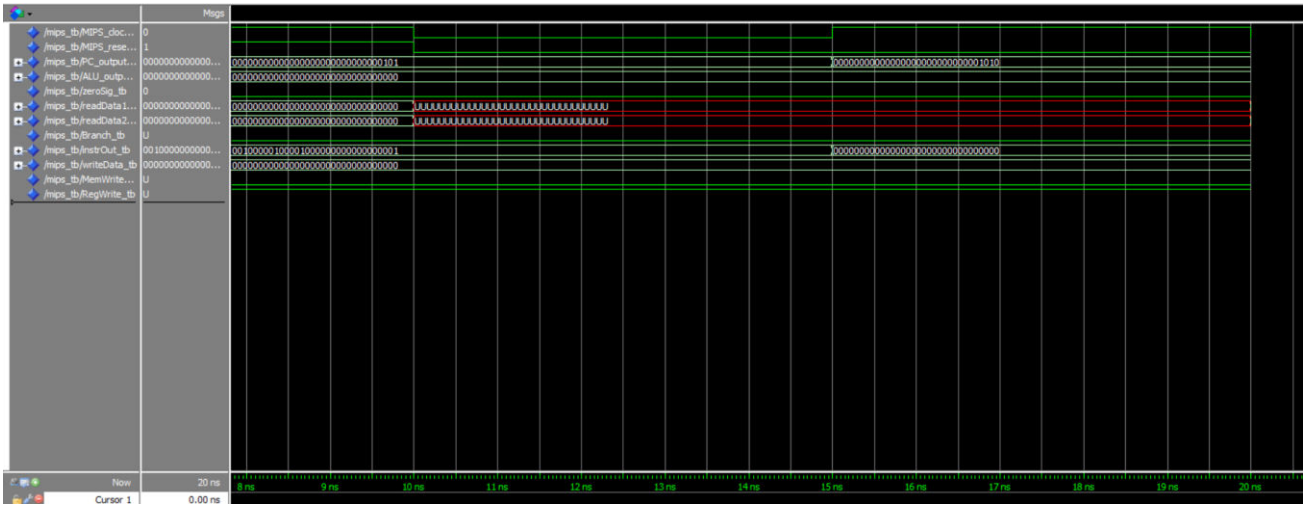
Before execution



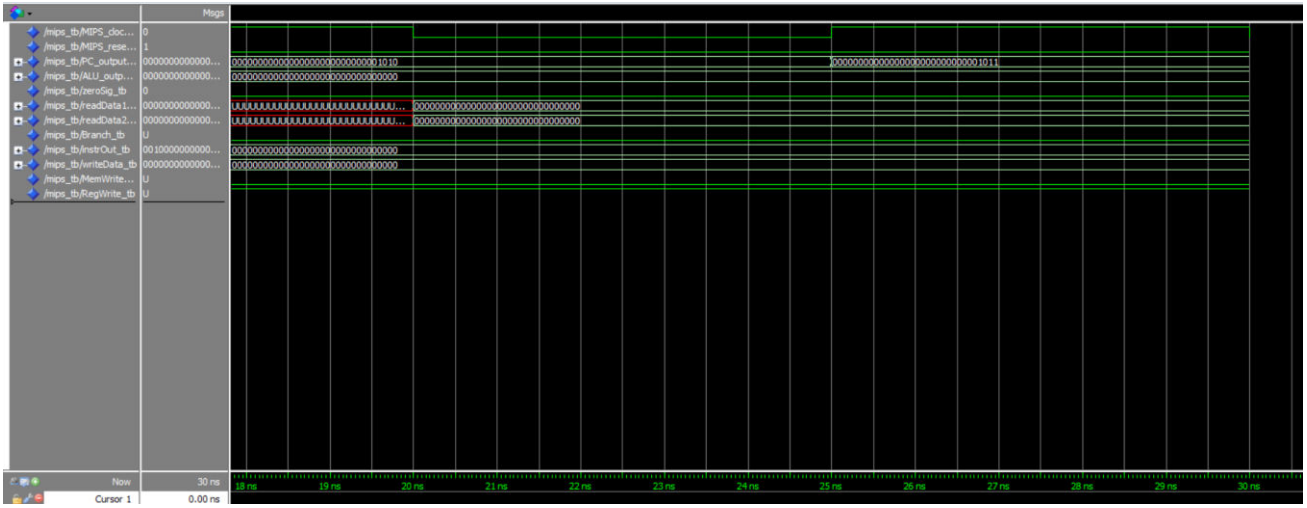
1 º clock



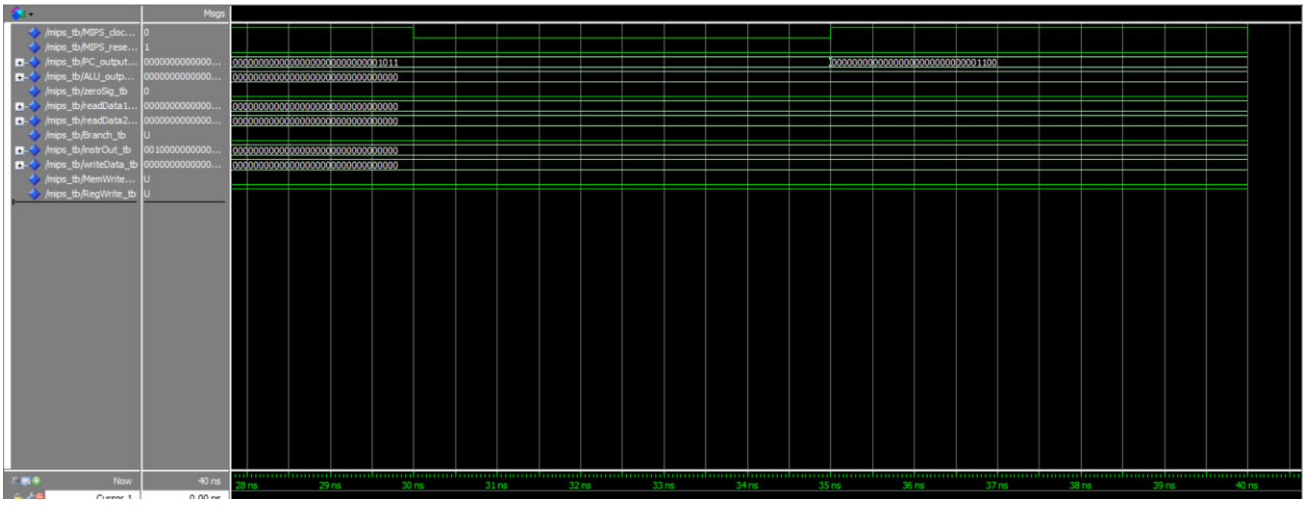
2 º clock



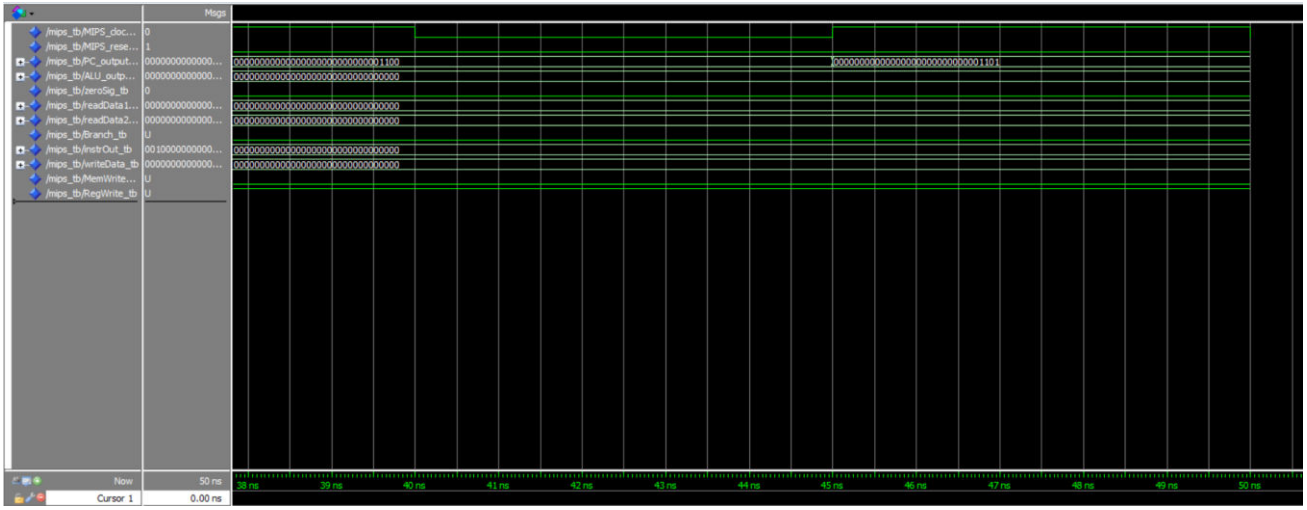
3 º clock



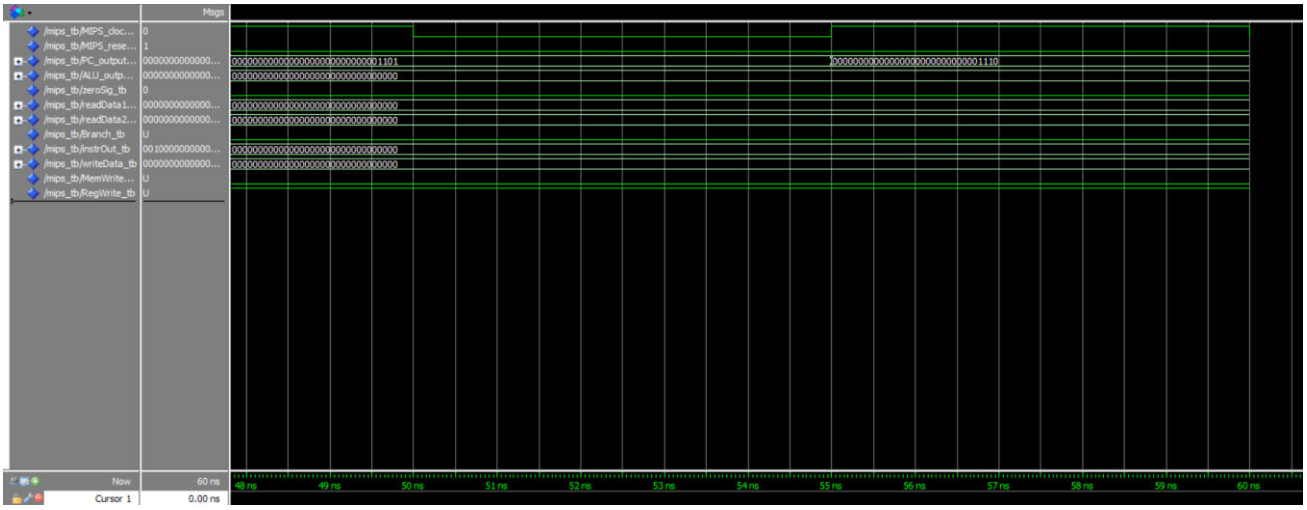
4 º clock



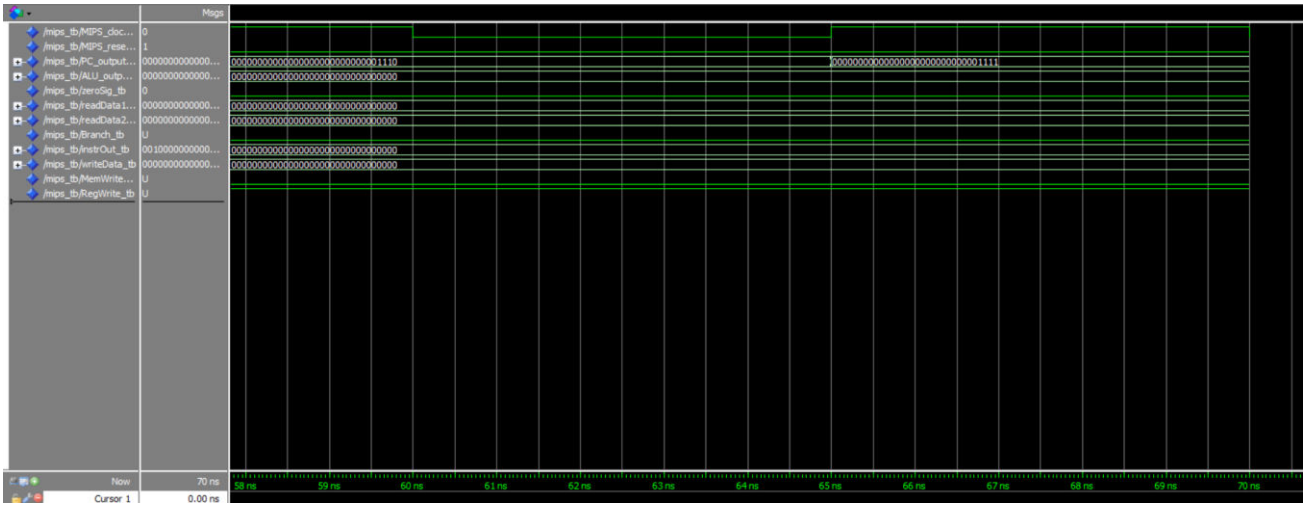
5 º clock



6 º clock



7 º clock



Thank you for your attention.